

Flex 4.5 code named "Hero"

It includes a few more Spark components, and will eventually support building mobile applications

Adobe previewing Melrose SDK

This can be used for developing for-pay application using Adobe AIR for sale in the Adobe marketplace and Intel's AppUp Store



Extending our AIR Feed Reader application with Flash Builder 4

Previously we created an RSS Feed reading application in Adobe AIR using Flash Builder 4. Our previous application had a simple UI which allowed us to enter a URL for the feed, and has two panes one below another. One to list the items in the feed, and the other to display the summary of the selected item.

We assume you have completed the previous part in this series (refer to page 104 of the September 2010 issue). Alternatively, you can check out the article at http://www.thinkdigit.com/d/fb4_tut1_4.

Our previous application could only handle a single feed at a time, which is a rather huge limitation for a feed reading application. In this article we will develop our application further by adding support for reading multiple feeds. We will make a Feed browser of sorts which will display a "tab bar" for switching between active feeds. Each "tab" will display its own list of feed items, and we will be able to select any item read the summary.

Understanding the changes to our application

So how do we go about this exactly? Let us first take into account the visual additions, in the form of controls / components. We will need:

A tab bar

We will simply use a button bar for this. A ButtonBar component makes it easy to add multiple buttons in a row. Each button will have a label corresponding to the title of the feed.

A button to add a new feed

Since we have to handle multiple feeds now, we need to have a button for adding a new feed. We will re-purpose the "Load" button of our previous application for this.



The interface is rather clean

A close button

We should have some way to remove the currently active tab if we want. This could be a simple button labelled "X" or if you wish to make it look better you can create an icon for it. That's about it; this will cover our needs from this application.

Now about the changes inside the application. Since we are now going to handle multiple feeds at the same time, we will need to have an array holding data for all the feeds, so it is ready when we click on it.

To wire everything so that it works, with minimal coding we will bind the array of feeds to the ButtonBar. This will automatically ensure that the feeds and tabs are in sync. We will also bind the feed data for active tab to the list showing the data items. This way the data will automatically change as we change the current tab.

The HTML component is already bound to this list, so that bit will work as always. What we need to do is to write the code that will add tabs to this array of tabs and load the corresponding data. We also need code for closing a tab and removing that feed from the array. The rest will work by itself!

The UI

Our UI is not changing drastically. Our load button will now transform to a

button labelled, "Add Feed". Our previous code was simply a horizontal group with a text input for the URL and a load button. Right after this we will now add the following:

```
<s:HGroup width="100%">
  <s:ButtonBar id="feedTabs"
```

```
requireSelection="true"
  width="100%" />
  <s:Button label="X" />
</s:HGroup>
```

This is another horizontal group with two components. The first is a ButtonBar which will display our tabs and let us switch between them. We have set its width to 100% so it takes up as much space as is available and pushes the close button to the edge. The second is the button to close the currently active tab, labelled simply "X". The requireSelection="true" bit in the code is to ensure that at least one tab is selected at any given time (if there is at least one tab of course). This is all we need to add, however the old "Load" button may be renamed to "Add Feed" or simply "+" if required.

The code

First of all we will define a new data type for the data associated with our feeds. For each feed tab, we will need to store a title and the feed data at least. Since this is a simple example, we will not manage refreshing the feeds every few minutes, otherwise it would have been wise to store the feed URL as well.

For this we will define a new ActionScript 3 class. This is simple to do with Flash Builder 4; simply right-click on your project, and under the "New" menu select ActionScript Class. This will pop up a new dialog box wherein you can enter details of your new class. Here is a little about some of the relevant parameters in the dialog: